

Maritime CargoService - Sprint 2

Andrea Passini, Paolo Savino, Alessandra Tolomelli

Luglio 2026

1 Introduzione

Lo Sprint 2 parte dal [prototipo](#) realizzato nello [Sprint 1](#), nel quale il ciclo di carico viene gestito tramite attori QAK, quali: IOPort, sonar e LED.

Come già anticipato, il goal di questo sprint è quello di

- Gestire e integrare i componenti sonar e led al Pico W fisico;
- Realizzare la porta IOPort come Web-GUI.
- Integrare l'attesa di 30 secondi tra l'accettazione della richiesta e il rilevamento del container sull'IOPort.
- Implementare il comportamento del marker device.

2 Analisi dei Requisiti

I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#)

3 Analisi del Problema

Lo Sprint 2 eredita dallo Sprint 1 un prototipo funzionante ma limitato e simulato, dove gli attori comunicano tra loro senza mai confrontarsi con vincoli reali di tempo, hardware o interazione umana. Il goal dello sprint introduce problemi distinti che è necessario analizzare separatamente prima di procedere alla progettazione.

3.1 Scelta Architetturale: MQTT

L'analisi del sistema di comunicazione evidenzia la necessità di far cooperare entità eterogenee caratterizzate da requisiti operativi, stack tecnologici e vincoli computazionali profondamente differenti.

Un approccio di comunicazione basato su invocazioni sincrone, come HTTP o REST, introdurrebbe un forte accoppiamento temporale e spaziale tra questi attori, riducendo la tolleranza ai guasti, ostacolando la scalabilità e costringendo i componenti QAK a gestire direttamente la logica di connessione verso client non nativi.

Per mitigare tali criticità, il sistema di comunicazione è stato consolidato attorno a un unico broker message-oriented, identificato in **Eclipse Mosquitto**, che implementa il protocollo **MQTT (Message Queuing Telemetry Transport)**. Questa scelta architetturale è motivata da diversi fattori:

- **Disaccoppiamento tramite Pattern Publish/Subscribe:** Il broker garantisce il completo disaccoppiamento spaziale e temporale tra i componenti software. I produttori pubblicano messaggi su *topic* logici, lasciando al broker il compito di instradarli ai sottoscrittori interessati.
- **Interoperabilità ed Estensibilità (Web-GUI):** L'adozione di uno standard aperto come MQTT, supportato nativamente anche su WebSockets, consente di innestare un client non nativo come il browser web direttamente nel flusso informativo del sistema. Ciò avviene in maniera trasparente, garantendo che l'introduzione o la modifica della Web-GUI non richieda alcuna variazione del codice o della configurazione degli attori QAK preesistenti.
- **Compatibilità con Pico W:** Essendo MQTT un protocollo estremamente leggero con un overhead di pacchetto ridotto al minimo, esso si adatta perfettamente alle limitazioni hardware del Raspberry Pi Pico W. L'integrazione sul microcontrollore è agevolata dall'uso della libreria nativa `umqtt.simple`, che permette di gestire la ricezione e l'invio dei comandi relativi al LED fisico senza saturare la memoria o compromettere la stabilità del firmware.

Il broker Mosquitto espone due listener distinti:

- **Porta 1883:** protocollo MQTT nativo (TCP), utilizzato dagli attori QAK (`cargoservice`, `sonar`, `led`, ecc.) e dal firmware MicroPython del Pico W tramite `umqtt.simple`;

- **Porta 9001:** listener MQTT-over-WebSocket, utilizzato per consentire alla Web-GUI di comunicare direttamente con il broker tramite la libreria `mqtt.js`.

Questa scelta è stata adottata per due motivi principali:

- i browser non supportano connessioni MQTT native su TCP, ma richiedono l'utilizzo del protocollo WebSocket.
- l'impiego di MQTT-over-WebSocket permette di mantenere un unico meccanismo di comunicazione publish/subscribe tra tutti i componenti del sistema, evitando l'introduzione di API HTTP dedicate o di componenti intermedi esclusivamente per la GUI.

Ciò rende l'architettura più uniforme, semplifica l'integrazione della Web-GUI e facilita eventuali estensioni future.

3.2 Marker-device e interazione con Cargoservice

Lo Sprint 1 aveva già previsto il **markerdevice** come QActor autonomo, inserito però come semplice scheletro logico privo di comportamento reale. Lo Sprint 2 ne definisce quindi il comportamento effettivo, ponendo il problema di stabilire con quale modalità debba interagire con **cargoservice** una volta posizionato il container su SLOT5.

Si è valutato se adottare un **Dispatch** (fire-and-forget), coerentemente con l'approccio già in uso per i segnali di stato non bloccanti del sistema (es. `led.blink`), oppure un modello **Request/Reply**. Il **Dispatch** è stato escluso perché il flusso del robot non può proseguire verso lo slot finale finché la marcatura non è stata effettivamente completata: si tratta di una dipendenza logica stretta, e non di un semplice aggiornamento di stato che può avvenire in modo asincrono. È stato quindi adottato il modello **Request/Reply**, che vincola esplicitamente **cargoservice** ad attendere la risposta `marking_done` prima di autorizzare il robot a dirigersi verso lo slot definitivo, riproducendo fedelmente il vincolo fisico secondo cui il container non può lasciare la postazione di etichettatura prima che l'operazione sia conclusa.

3.3 Analisi della Tecnologia: MicroPython e uMQTT

La scelta dello stack tecnologico da usare sul Raspberry Pi Pico W è stata guidata dalla necessità di bilanciare la semplicità di sviluppo con l'efficienza nell'integrazione di rete.

- **Perché MicroPython (rispetto a C/C++):** Sebbene il linguaggio C tramite Pico SDK offra le massime prestazioni hardware e il minor consumo di memoria, la sua curva di apprendimento e la complessità nella gestione dello stack Wi-Fi e dei socket TCP avrebbero introdotto una notevole complessità accidentale. L'adozione di MicroPython permette di astrarre completamente la gestione della rete.
- **Perché la libreria uMQTT:** Per implementare il protocollo MQTT su una piattaforma con risorse limitate come il Pico W, si è scelto di utilizzare `umqtt.simple`. Questa libreria rappresenta lo standard di fatto nell'ecosistema MicroPython: è estremamente leggera, ha un footprint di memoria ridotto al minimo e offre un'API sincrona e non bloccante, `check_msg()`.

3.4 Scelta del Sonar Logico come intermediario per HC-SR04

Il QActor sonar svolge il ruolo di sensore logico intermedio: riceve le richieste grezze, le elabora e le converte in eventi, sollevando il **cargoservice** dalle operazioni di filtraggio. In questo modo, il **cargoservice** mantiene il ruolo di orchestratore ad alto livello, rimanendo completamente **disaccoppiato** dal sensore sonar fisico. Questa soluzione favorisce inoltre la scalabilità del sistema, semplificando l'integrazione di eventuali QActor futuri.

3.5 Scelta del Led Logico come intermediario per l'attuatore fisico

Il QActor led agisce come componente di astrazione dell'hardware: riceve comandi di controllo concettuali (come accensione, spegnimento o lampeggio) e gestisce autonomamente il cambio di stato del dispositivo fisico. Questo approccio solleva il **cargoservice** dai dettagli implementativi del circuito, preservandone la natura di puro orchestratore e garantendo un completo **disaccoppiamento** dalla componente fisica. Inoltre,

l'interposizione di questo livello logico rende immediata l'eventuale sostituzione o estensione dell'hardware senza impattare sulla logica applicativa principale.

3.6 IOPort come Web-GUI

I requisiti impongono l'adozione di una **Web-GUI** in sostituzione all'interfaccia simulata **IOPort mock** dello Sprint precedente. Dal punto di vista dell'analista, questo requisito introduce la necessità di far cooperare il browser web con l'ecosistema QAK nativo ad attori. L'analisi si concentra sulle modalità ottimali per soddisfare questo vincolo tecnologico senza snaturare l'architettura distribuita.

3.6.1 Allineamento al Protocollo dei Messaggi

La Web-GUI deve poter inviare comandi che gli attori QAK siano in grado di comprendere ed elaborare nativamente. Il framework QAK impone una struttura rigida basata su stringhe.

```
msg(MSGID, MSGTYPE, SENDER, RECEIVER, CONTENT, SEQNUM)
```

Poiché la Web-GUI JavaScript non dispone delle librerie native Kotlin del framework per serializzare questi messaggi, si è dovuto analizzare come generare tale formato: si è deciso di implementare la serializzazione direttamente lato client in JavaScript. In questo modo, la Web-GUI comunica coi componenti nativi, riducendo a zero l'infrastruttura server-side necessaria.

3.6.2 Scelta del Canale di Comunicazione (MQTT-over-WebSockets)

Dal momento che la GUI deve essere reattiva esclude l'uso di tradizionali chiamate HTTP sincrone o polling. La soluzione che in analisi viene ritenuta ottimale consiste nello sfruttare il broker MQTT già consolidato nel sistema, esponendo una comunicazione sulla porta WebSocket. Questo permette al browser di:

- Stabilire una connessione persistente e bidirezionale direttamente con il broker.
- Ricevere gli aggiornamenti di stato in modalità *push* (asincrona) non appena gli attori QAK pubblicano sui rispettivi topic.
- Iniettare messaggi direttamente nei topic di input dei componenti, garantendo l'interoperabilità senza richiedere particolari modifiche o adattatori agli attori QAK.

3.7 Scelta di comunicazione tra IOPort e Cargoservice

La scelta di adottare una comunicazione diretta tra la GUI e il cargoservice, senza un QActor intermedio per l'IOPort, risulta la soluzione più appropriata per il contesto attuale. Questo approccio semplifica l'architettura, riduce il numero di componenti e mantiene bassa la latenza di comunicazione. Sebbene introduca un maggiore accoppiamento tra GUI e cargoservice, tale compromesso è accettabile considerando la presenza di una sola IOPort.

Un eventuale QActor dedicato potrà essere introdotto in futuro qualora il sistema debba gestire più IOPort o logiche di comunicazione più complesse.

4 Piano di Lavoro e Distribuzione dei Compiti

#	Task	Assegnatario	Ore Stimate
1	Analisi del problema	Savino	5
1	TestPlan	Savino	4
1	Implementazione effettiva marker-device	Tolomelli	2
2	Implementazione della logica ad eventi del sonar	Passini, Tolomelli	3
3	Integrazione del LED del Pico W	Passini	4
5	Implementazione dell'IOPort come Web-GUI	Passini	5
6	Revisione	Tutti	6

5 TestPlan

I test descritti in questa sezione verificano puntualmente i comportamenti analizzati nella Sezione 3 (Analisi del Problema): connessione al broker, ciclo di carico end-to-end, filtraggio del sonar e comando del LED fisico. I test sono eseguiti pubblicando/sottoscrivendo manualmente messaggi MQTT tramite i client a riga di comando `mosquitto_pub/mosquitto_sub`, in modo da isolare ciascun attore dal resto del sistema.

5.1 Connessione al broker

Verifica che entrambi i listener del broker Mosquitto siano attivi e raggiungibili, replicando sia il canale nativo usato dagli attori QAK sia quello WebSocket usato dalla Web-GUI.

```
# Canale nativo MQTT (porta 1883) - usato dagli attori QAK e dal Pico W
mosquitto_sub -h localhost -p 1883 -t "unibo/qak/#" -v

# Canale WebSocket (porta 9001) - usato dalla Web-GUI ioport.html
mosquitto_sub -h localhost -p 9001 -t "sprint2/#" -v
```

Esito atteso: entrambe le sottoscrizioni restano attive senza errori di connessione; eventuali messaggi pubblicati dagli attori appaiono in tempo reale su console.

5.2 Flusso di richiesta di carico

Simula la pressione del pulsante Pushbutton sulla Web-GUI, inviando direttamente il messaggio di richiesta a `cargoservice`.

```
mosquitto_pub -h localhost -p 1883 -t "unibo/qak/cargoservice" \
-m "msg(load_request,request,ioport,cargoservice,loadRequest(none),1)"
```

Esito atteso: con hold non piena, `cargoservice` risponde con `loadAccepted(SLOTID)` e il LED passa in stato lampeggiante (`ledBlink("on")`); con hold piena, risponde `loadRefused(none)`.

5.3 Timeout dei 30 secondi

Verifica il rilascio automatico dello slot se il container non viene posato entro il tempo limite dopo l'accettazione della richiesta (Test 2), senza inviare alcun messaggio di posa container.

```
# Dopo aver ricevuto loadAccepted, attendere 30s senza pubblicare
# alcun messaggio su unibo/qak/sonar, poi osservare l'evento:
mosquitto_sub -h localhost -p 1883 -t "sprint2/#" -v
# Atteso entro 30s: loadTimeout(none) + ledBlink("off") + releaseSlot
```

Esito atteso: dopo 30000 ms viene emesso `loadTimeout(none)`, lo slot torna `free` nell'attore `hold` e il LED si spegne.

5.4 Filtraggio ad eventi del sonar (3s di stabilità)

Simula la posa del container sulla pedana, inviando il dispatch direttamente al `sonar` come fa `onContainerPlaced()` nella Web-GUI.

```
mosquitto_pub -h localhost -p 1883 -t "unibo/qak/sonar" \
  -m "msg(container_on_platform,dispatch,ioport,sonar,containerOnPlatform(none),2)"
```

Esito atteso: in console compaiono 3 log consecutivi (“Container stabile per 1/2/3 secondi...”), seguiti dall’emissione di `containerDetected(none)`; il robot viene quindi richiesto verso `IOPort`.

5.5 Comando LED fisico sul Pico W

Verifica indipendente del componente fisico, bypassando l’attore QAK e pubblicando direttamente sul topic letto da `led_blink.py`.

```
# ON: il LED deve iniziare a lampeggiare ogni 500ms
mosquitto_pub -h <BROKER_IP> -p 1883 -t "cargo/led/cmd" -m "on"

# OFF: il LED deve spegnersi immediatamente
mosquitto_pub -h <BROKER_IP> -p 1883 -t "cargo/led/cmd" -m "off"
```

Esito atteso: il LED integrato del Pico W lampeggia con periodo di 500ms dopo il comando `on`, e si spegne istantaneamente dopo il comando `off` (verifica del fix sull’atomo quotato descritto nella Sez. 3.4).

6 Progettazione

6.1 Implementazione di MQTT

Nel modello QAK, il broker e il topic di sistema vengono dichiarati direttamente nella direttiva di intestazione, consentendo alla runtime di sottoscrivere automaticamente ai messaggi dell’infrastruttura.

```
System cargosystem

// Allineato all'indirizzo localhost per la connessione broker locale

mqttBroker "localhost" : 1883   eventTopic "sprint2"
```

Lato client web, la connessione viene istanziata programmaticamente al caricamento della pagina: la libreria JavaScript esegue l’handshake WebSocket e procede subito alla registrazione dei topic per la ricezione asincrona degli aggiornamenti dal sistema.

```

const brokerUrl = "ws://localhost:9001";
let client = null;

function autoConnect() {
  client = mqtt.connect(brokerUrl);

  client.on('connect', function () {
    setDisplayText("PRONTO - Premere il pulsante per caricare");
    client.subscribe('sprint2/#');
    client.subscribe('unibo/qak/ioport');
  });

  client.on('message', function (topic, message) {
    const payload = message.toString();
    parseIncomingMessage(payload);
  });
}

```

6.2 Implementazione markerdevice

Coerentemente con quanto stabilito in fase di analisi, il **markerdevice** è stato implementato come QActor indipendente, che riceve una richiesta di marcatura, simula il tempo di elaborazione fisico dell'etichettatrice tramite un `delay`, e risponde con l'esito dell'operazione.

```

QActor markerdevice context ctxcargo {
  State s0 initial {
    println("markerdevice | avviato") color blue
  }
  Goto idle

  State idle {
    println("markerdevice | idle") color green
  }
  Transition t0
    whenRequest do_marking -> handle_marking

  State handle_marking {
    println("markerdevice | Ricevuto comando di marcatura su SLOT5.
    Elaborazione barcode in corso...") color yellow
    delay 2000
    println("markerdevice | Marcatura completata! Barcode applicato correttamente
    sul container.") color green
    replyTo do_marking with marking_done : markingDone(none)
  }
  Goto idle
}

```

Sul lato dell'orchestratore, l'interazione avviene in modalità **request/reply**, bloccando l'avanzamento logico del robot finché non viene ricevuta la risposta `marking_done`.

```

State request_marking_activity {
    println("cargoservice | Robot positioned on SLOT5. Requesting marking to
    markerdevice...") color green
    request markerdevice -m do_marking : doMarking(none)
}
Transition t5
    whenReply marking_done -> go_to_final_slot

```

6.3 Integrazione del LED Pi Pico W

L'attore **led** funge da proxy verso il componente fisico: riceve i dispatch **led_blink** dall'orchestratore e ne inoltra lo stato pubblicandolo sul topic MQTT **cargo/led/cmd**, letto direttamente dal firmware MicroPython in esecuzione sul Pico W.

```

QActor led context ctxcargo {
    [# var ledState = "off" #]

    State off {
        println("led | LED SPENTO") color green
    }
    Transition t0
        whenMsg led_blink -> handle_initial_blink

    State handle_initial_blink {
        onMsg( led_blink : ledBlink(STATE) ) {
            [#
                ledState = payloadArg(0)
                mqtt.publish("cargo/led/cmd", ledState)
            #]
        }
    }
    Goto on_blink if [# ledState == "on" #] else off
}

```

Sul lato fisico, il Pico W esegue uno script MicroPython, [led.sonar.py](#), che si connette al WiFi e allo stesso broker Mosquitto (porta 1883), sottoscrivendo il topic **cargo/led/cmd**. La lettura dei messaggi è non bloccante (**check_msg()**), così da poter gestire il lampeggio del LED con un timer software a 500ms senza fermare il loop principale.


```

BROKER_IP = "10.158.187.242" # IP reale del PC/server
TOPIC = b"cargo/led/cmd"
BLINK_INTERVAL_MS = 500

def on_msg(topic, msg):
    global blinking
    if msg == b"on":
        blinking = True
    elif msg == b"off":
        blinking = False
        led.value(0)

client = MQTTClient("pico_led", BROKER_IP, 1883)
client.set_callback(on_msg)
client.connect()
client.subscribe(TOPIC)

while True:
    client.check_msg()
    if blinking:
        now = time.ticks_ms()
        if time.ticks_diff(now, last_toggle) >= BLINK_INTERVAL_MS:
            led_phys_state = not led_phys_state
            led.value(led_phys_state)
            last_toggle = now
        time.sleep_ms(50)

```

6.4 IOPort come Web-GUI

L'IOPort mock dello Sprint 1 è stato sostituito da un'interfaccia web, [web-gui.html](#), che riproduce il pannello fisico della plancia di carico: un pulsante di richiesta (pushbutton), un display di stato ed un sensore di posa container, tutti pilotati esclusivamente via MQTT-over-WebSocket (porta 9001), senza alcuna logica applicativa lato server.

Alla pressione del pulsante, la pagina costruisce un messaggio nel formato atteso dal sistema QAK e lo pubblica sul topic di richiesta verso **cargoservice**.

```

function buildQakMessage(msgId, msgType, sender, receiver, content) {
    return 'msg(${msgId},${msgType},${sender},${receiver},${content},${msgSeqNum++})';
}

function onPushbuttonPressed() {
    setDisplayText("INVIO RICHIESTA IN CORSO...");
    const qakMsg = buildQakMessage("load_request", "request", "ioport", "cargoservice",
    , "loadRequest(none)");

    if (client && client.connected) {
        client.publish("unibo/qak/cargoservice", qakMsg);
    } else {
        setDisplayText("ERRORE: RETE ASSENTE");
    }
}

```

In modo analogo, la posa del container invia un dispatch diretto al sonar, disabilitando immediatamente il pulsante per evitare invii multipli durante la finestra di filtraggio dei 3 secondi.

```
function onContainerPlaced() {
  setDisplayText("CONTAINER POSIZIONATO - ATTESA FILTRAGGIO SONAR...");
  enableSensorButton(false);

  const qakMsg = buildQakMessage("container_on_platform", "dispatch", "ioport"
    , "sonar", "containerOnPlatform(none)");
  if (client && client.connected) {
    client.publish("unibo/qak/sonar", qakMsg);
  }
}
```

Infine, la funzione `parseIncomingMessage` interpreta i messaggi in arrivo dal sistema (accettazione, timeout, rifiuto, completamento) e aggiorna di conseguenza il display e lo stato dei controlli, realizzando così un feedback in tempo reale coerente con lo stato della macchina a stati di **cargoservice**.

```
function parseIncomingMessage(payload) {
  if (payload.includes('loadAccepted')) {
    const match = payload.match(/loadAccepted\((\w+)\)/);
    currentReservedSlot = match[1];
    setDisplayText('RICHIESTA ACCETTATA! RISERVATO:
      ${currentReservedSlot.toUpperCase()}...');
    enableSensorButton(true);
  }
  else if (payload.includes('loadTimeout')) {
    setDisplayText("TIMEOUT: TEMPO SCADUTO (30s)! RICHIESTA ANNULLATA.");
    enableSensorButton(false);
  }
  else if (payload.includes('loadRefused')) {
    setDisplayText("RICHIESTA RIFIUTATA: STIVA COMPLETA.");
  }
  else if (payload.includes('robotCompleteNotif')) {
    const match = payload.match(/robotCompleteNotif\((\w+)\)/);
    setDisplayText('STIVAGGIO COMPLETATO CON SUCCESSO IN ${match[1].toUpperCase()}');
  }
}
```

6.5 Integrazione Sensore Sonar HC-SR04: MicroPython e sonar logico

Dal punto di vista operativo, il Pico W legge l'**HC-SR04** via `time_pulse_us()` e pubblica le distanze su MQTT ogni 300ms; il QActor sonar, invece, riceve le letture tramite Dispatch `sensor_data`.

Due condizioni vengono rilevate: **container presente** ($D < 50\text{cm}$ per 10 letture consecutive $\approx 3\text{s}$) e **sensore guasto** ($D > 100\text{cm}$ per 10 letture).

La comunicazione verso cargoservice avviene esclusivamente via Event (`container_detected`, `sonar_fault`), garantendo disaccoppiamento totale. Questo design permette sia letture reali da HC-SR04 che trigger manuale da GUI per test senza hardware.

6.5.1 Lettura Hardware e Pubblicazione MQTT (Pico W)

```
def read_distance_cm():
    TRIG.value(0)
    time.sleep_us(2)
    TRIG.value(1)
    time.sleep_us(10)
    TRIG.value(0)
    try:
        duration = machine.time_pulse_us(ECHO, 1, 25000)
    except OSError:
        return 120.0
    if duration < 0:
        return 120.0
    return duration / 58.0

# Ciclo principale: ogni 300ms
if time.ticks_diff(now, last_sonar_publish) >= 300:
    dist = read_distance_cm()
    qak_msg = f"msg(sensor_data,dispatch,pico_sonar,sonar,sensorData({dist:.1f}},{seq})"
    client.publish("unibo/qak/sonar", qak_msg)
    last_sonar_publish = now
```

6.5.2 Logica di Debounce con Contatori Indipendenti (QActor)

```
QActor sonar context ctxcargo {
[#
    ...
#]

State process_sensor_data {
    onMsg( sensor_data : sensorData(DISTANCE) ) {
        [# lastDistance = payloadArg(0).toString().toDouble() #]
    }
    [#
        if (lastDistance < DETECT_THRESHOLD) {
            ContainerCounter = minOf(ContainerCounter + 1, STABLE_READINGS)
            FaultCounter = maxOf(FaultCounter - 1, 0)
        }
        else if (lastDistance > DFREE) {
            FaultCounter = minOf(FaultCounter + 1, STABLE_READINGS)
            ContainerCounter = maxOf(ContainerCounter - 1, 0)
        }
        else {
            ContainerCounter = maxOf(ContainerCounter - 1, 0)
            FaultCounter = maxOf(FaultCounter - 1, 0)
        }
    #]
}
Goto emit_event_container if [# ContainerCounter >= STABLE_READINGS #]
else check_fault_emit

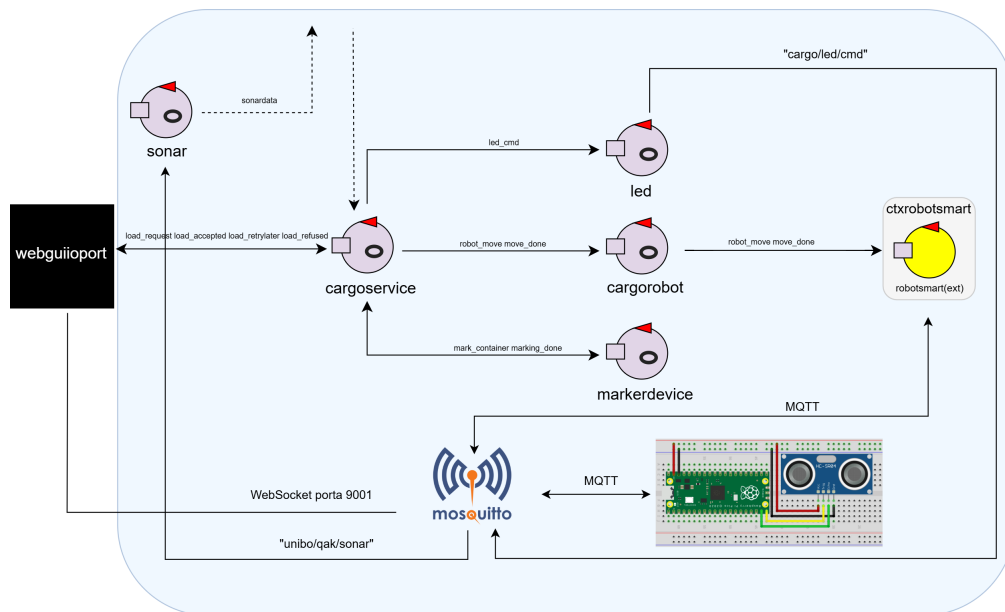
State emit_event_container {
    emit container_detected : containerDetected(none)
    [# ContainerCounter = 0; FaultCounter = 0 #]
}
Goto idle

State emit_event_fault {
    emit sonar_fault : sonarFault(none)
    [# ContainerCounter = 0; FaultCounter = 0 #]
}
Goto idle
}
```

7 Sintesi

7.1 Architettura del sistema

Ecco una pagina di sintesi che riporta l'architettura finale del [prototipo](#).



7.2 Istruzioni sul Deployment

Per procedere al deployment del prototipo è necessario effettuare i passaggi qui riportati:

1. Clonare il repository del progetto da [GitHub](#).
2. Nella directory **robotsmart26/yamls** eseguire il comando `docker-compose -f unibobasic26.yaml up`.
3. Aprire un browser e andare su `http://localhost:8090/`.
4. Eseguire all'interno della directory del progetto **robotsmart26** il comando `./gradlew run`.
5. Eseguire all'interno della directory del progetto **sprint2/yamls** il comando `docker-compose -f mosquitto.yml -p mosquitto up`.
6. Eseguire all'interno della directory del progetto **sprint2** il comando `./gradlew run`.
7. Nella directory **sprint2/web** aprire **ioport.html**.
8. Collegare il Pico W al computer e nella directory **sprint2/python**, modificare **led_blink.py** con le informazioni relative al proprio Wi-Fi connesso (SSID e password) e il **broker_ip** con l'IP reale del proprio PC/server.